

Build Your First Playwright Test: Login and Verify with Assertions

Objective

This activity equips you to modernize UI testing with Playwright—focusing on speed, reliability, and user-centric validation. You'll build a robust login test using semantic selectors and explore tools for debugging and reporting. Are your tests simulating real behavior? Are they stable enough for CI? This is your first step toward scalable, modern test automation.

Tools used: Playwright

Scenario

You're the lead test automation engineer at a fast-growing SaaS company offering project management software to distributed teams. The product team is expanding rapidly, and quality is becoming a bottleneck.

The current QA process suffers from

- * Slow and flaky UI tests
- * Missed issues in login flows
- * Low confidence in test coverage

Your manager has asked you to modernize UI testing using Playwright, starting with a robust login flow and dashboard validation.

Your Mission

Set up a Playwright project and build a login test that:

- * Uses semantic selectors

- * Simulates a real user session

- * Validates successful login and dashboard visibility

Instructions for learners

Task 1 Install and Bootstrap Playwright

1. Create a new folder and initialize a project:

`mkdir project-test-suite`

`cd project-test-suite`

`npm init -y`

`npm install -D @playwright/test`

`npx playwright install`

2. Scaffold Playwright's default project structure:

↳ `npx playwright test --init`

You should now have:

↳ * `playwright.config.ts`

↳ * `tests/` folder with a sample test

Task 2 Create a Login Test**

→ Use a real or public test app.

→ Recommended:

<https://www.saucedemo.com>

Credentials:

→ Username: standard_user

→ Password: secret_sauce

Create a file at tests/login.spec.ts with the following content:

import { test, expect } from '@playwright/test';

```
test('user can log in and see dashboard', async ({ page }) => {  
  await page.goto('[https://www.saucedemo.com](https://www.saucedemo.com)');  
  await page.getByPlaceholder('Username').fill('standard_user');  
  await page.getByPlaceholder('Password').fill('secret_sauce');  
  await page.getByRole('button', { name: 'Login' }).click();  
  await expect(page).toHaveURL(/inventory/);  
  await expect(page.getByText('Products')).toBeVisible();  
});
```

Task 3 Run and Debug the Test

Run the test:

`npx playwright test`

If it fails, use UI mode for debugging:

`npx playwright test --ui`

Tip: Use `.pause()` or enable tracing for deeper debugging.

Task 4 (Optional Bonus) Add Trace and Screenshot Reporting

→ In playwright.config.ts, enable debugging artifacts:

→
use: {
 trace: 'on',
 screenshot: 'only-on-failure',
}

→ 7. After running tests, view the report:

→ npx playwright show-report

Completion

Criteria

- * Playwright is successfully installed and initialized
- * Login test is implemented using semantic selectors (getByRole, getByPlaceholder)
- * The test passes and asserts key dashboard elements
- * Trace and report artifacts are generated on failure
- * Learner explores debugging and inspection via Playwright UI mode

Reflection

Reflection Prompts

Use Coursera Coach or team discussion to reflect on:

- * What made Playwright easier than previous tools (e.g., Selenium, Cypress)?
- * How did auto-waiting and semantic selectors improve test stability?
- * How might you extend this test to support multi-role login using browser contexts?
- * How would you integrate this test into a CI/CD pipeline?